

Advanced/Enterprise Java

Agenda

- JDBC

Java Database Connectivity (JDBC)

- RDBMS understand SQL language only.
- JDBC driver converts Java requests in database understandable form and database response in Java understandable form.
- JDBC drivers are of 4 types
 - Type I - Jdbc Odbc Bridge driver
 - ODBC is standard of connecting to RDBMS (by Microsoft).
 - Needs to create a DSN (data source name) from the control panel.
 - From Java application JDBC Type I driver can communicate with that ODBC driver (DSN).
 - The driver class: `sun.jdbc.odbc.JdbcOdbcDriver` -- built-in in Java.
 - database url: `jdbc:odbc:dsn`
 - Advantages:
 - Can be easily connected to any database.
 - Disadvantages:
 - Slower execution (Multiple layers).
 - The ODBC driver needs to be installed on the client machine.
 - Type II - Partial Java/Native driver
 - Partially implemented in Java and partially in C/C++. Java code calls C/C++ methods via JNI.
 - Different driver for different RDBMS. Example: Oracle OCI driver.
 - Advantages:
 - Faster execution
 - Disadvantages:
 - Partially in Java (not truly portable)
 - Different driver for Different RDBMS
 - Type III - Middleware/Network driver
 - Driver communicate with a middleware that in turn talks to RDBMS.
 - Example: WebLogic RMI Driver
 - Advantages:
 - Client coding is easier (most task done by middleware)
 - Disadvantages:
 - Maintaining middleware is costlier
 - Middleware specific to database
 - Type IV
 - Database specific driver written completely in Java.
 - Fully portable.
 - Most commonly used.
 - Example: Oracle thin driver, MySQL Connector/J, ...

MySQL Programming Steps

- step 0: Add JDBC driver into project/classpath.
 - Project Properties -> Java Build Path -> Libraries - Classpath -> Add External Jar -> select mysql driver jar -> Ok
- step 1: Load and register JDBC driver class. These drivers are auto-registered when loaded first time in JVM. This step is optional in Java SE applications from JDBC 4 spec.

```
Class.forName("com.mysql.cj.jdbc.Driver");  
// for Oracle: Use driver class oracle.jdbc.driver.OracleDriver
```

- step 2: Create JDBC connection using helper class DriverManager.

```
// db url = jdbc:dbname://db-server:port/database  
Connection con =  
DriverManager.getConnection("jdbc:mysql://localhost:3306/dbname", "root",  
"manager");  
// for Oracle: jdbc:oracle:thin:@localhost:1521:sid
```

- step 3: Create the statement.

```
Statement stmt = con.createStatement();
```

- step 4: Execute the SQL query using the statement and process the result.

```
String sql = "non-select query";  
int count = stmt.executeUpdate(sql); // returns number of rows affected
```

- OR

```
String sql = "select query";  
ResultSet rs = stmt.executeQuery(sql);  
while(rs.next()) // fetch next row from db (return false when all rows  
completed)  
{  
    x = rs.getInt("col1"); // get first column from the current row  
    y = rs.getString("col2"); // get second column from the current row  
    z = rs.getDouble("col3"); // get third column from the current row  
    // process/print the result  
}  
rs.close();
```

- step 5: Close statement and connection.

```
stmt.close();
con.close();
```

MySQL Driver Download

- <https://mvnrepository.com/artifact/com.mysql/mysql-connector-j/8.4.0>

SQL Injection

- Building queries by string concatenation is inefficient as well as insecure.
- Example:

```
dno = sc.nextLine();
sql = "SELECT * FROM emp WHERE deptno="+dno;
```

- If user input "10", then effective SQL will be "SELECT * FROM emp WHERE deptno=10". This will select all emps of deptno 10 from the RDBMS.
- If user input "10 OR 1", then effective SQL will be "SELECT * FROM emp WHERE deptno=10 OR 1". Here "1" represent true condition and it will select all rows from the RDBMS.
- In Java, it is recommended NOT to use "Statement" and building SQL by string concatenation. Instead use PreparedStatement.

PreparedStatement

- PreparedStatement represents parameterized queries.

```
String sql = "SELECT * FROM students WHERE name=?";
PreparedStatement stmt = con.prepareStatement(sql);
System.out.print("Enter name to find: ");
String name = sc.next();
stmt.setString(1, name);
ResultSet rs = stmt.executeQuery();
while(rs.next()) {
    int roll = rs.getInt("roll");
    String name = rs.getString("name");
    double marks = rs.getDouble("marks");
    System.out.printf("%d, %s, %.2f\n", roll, name, marks);
}
```

- The same PreparedStatement can be used for executing multiple queries. There is no syntax checking repeated. This improves the performance.

MySQL Programming steps -- PreparedStatement

1. Add JDBC driver into project/classpath.
 - Java project -> Properties -> Java Build Path -> Libraries -> Add External Jars -> select MySQL JDBC driver jar -> Apply and Close.
2. Load and register driver class.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

3. Create database connection.

```
Connection con =  
DriverManager.getConnection("jdbc:mysql://localhost:3306/dbname", "dbuser",  
"dbpassword");
```

4. Create PreparedStatement with (parameterized) SQL query.

```
String sql = "sql query with ?";  
PreparedStatement stmt = con.prepareStatement(sql);
```

5. Set param values, execute the query and process the result.

```
stmt.setInt(1, val1); // set 1st param ? value  
stmt.setString(2, val2); // set 2nd param ? value
```

```
// for non-SELECT queries  
int count = stmt.executeUpdate();
```

```
// for SELECT queries  
ResultSet rs = stmt.executeQuery();  
while(rs.next()) {  
    int val1 = rs.getInt("col1");  
    String val2 = rs.getString("col2");  
    // ...  
}  
rs.close();
```

6. Close statement and connection.

```
stmt.close();  
con.close();
```

JDBC concepts

java.sql.Driver

- Implemented in JDBC drivers.
 - MySQL: com.mysql.cj.jdbc.Driver
 - Oracle: oracle.jdbc.OracleDriver
 - Postgres: org.postgresql.Driver
- Driver needs to be registered with DriverManager before use.
- When driver class is loaded, it is auto-registered (Class.forName()).
- Driver object is responsible for establishing database "Connection" with its connect() method.
- This method is called from DriverManager.getConnection().

java.sql.Connection

- Connection object represents database socket connection.
- All communication with db is carried out via this connection.
- Connection functionalities:
 - Connection object creates a Statement.
 - Transaction management.

java.sql.Statement

- Represents SQL statement/query.
- To execute the query and collect the result.

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery(selectQuery);
```

```
int count = stmt.executeUpdate(nonSelectQuery);
```

- Since query built using string concatenation, it may cause SQL injection.

java.sql.PreparedStatement

- Inherited from java.sql.Statement.
- Represents parameterized SQL statement/query.
- The query parameters (?) should be set before executing the query.
- Same query can be executed multiple times, with different parameter values.
- This speed up execution, because query syntax checking is done only once.

```
PreparedStatement stmt = con.prepareStatement(query);
```

```
stmt.setInt(1, intValue);  
stmt.setString(2, stringValue);  
stmt.setDouble(3, doubleValue);  
stmt.setDate(4, dateObject); // java.sql.Date  
stmt.setTimestamp(5, timestampObject); // java.sql.Timestamp
```

```
ResultSet rs = stmt.executeQuery();  
// OR  
int count = stmt.executeUpdate();
```

java.sql.ResultSet

- ResultSet represents result of SELECT query. The result may have one/more rows and one/more columns.
- Can access only the columns fetched from database in SELECT query (projection).

```
// SELECT id, quote, created_at FROM quotes  
ResultSet rs = stmt.executeQuery();  
while(rs.next()) {  
    int id = rs.getInt("id");  
    String quote = rs.getString("quote");  
    Timestamp createdAt = rs.getTimestamp("created_at"); //  
    java.sql.Timestamp  
    // ...  
}
```

```
// SELECT id, quote, created_at FROM quotes  
ResultSet rs = stmt.executeQuery();  
while(rs.next()) {  
    int id = rs.getInt(1);  
    String quote = rs.getString(2);  
    Timestamp createdAt = rs.getTimestamp(3); // java.sql.Timestamp  
    // ...  
}
```

Quick Revision

Statements

- interface CreateStatement: executing SQL queries
 - Drawback: Prepare queries by String concatenation. May cause SQL injection.
- interface PreparedStatement extends Statement: executing parameterized SQL queries
 - Prevent SQL injection
 - Efficient execution if same query is to be executed repeatedly.
- interface CallableStatement extends PreparedStatement: executing stored procedures in db
 - Prevent SQL injection
 - More efficient execution if same query is to be executed repeatedly.
- Executing SELECT statements

```
try(Connection con = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
    String sql = "SELECT * FROM students WHERE marks > ?";
    try(PreparedStatement stmt = con.prepareStatement(sql)) {
        stmt.setDouble(1, marks);
        try(ResultSet rs = stmt.executeQuery()) {
            while(rs.next()) {
                int roll = rs.getInt("roll");
                String name = rs.getString("name");
                double smarks = rs.getDouble("marks");
                Student s = new Student(roll, name, marks);
                System.out.println(s);
            }
        } // rs.close()
    } // stmt.close()
} // con.close()
catch(Exception ex) {
    ex.printStackTrace();
}
```

- Executing non-SELECT statements

```
try(Connection con = DriverManager.getConnection(DB_URL, DB_USER,
DB_PASSWORD)) {
    String sql = "DELETE FROM students WHERE marks > ?";
    try(PreparedStatement stmt = con.prepareStatement(sql)) {
        stmt.setDouble(1, marks);
        int count = stmt.executeUpdate();
        System.out.println("Rows Deleted: " + count);
    } // stmt.close()
} // con.close()
catch(Exception ex) {
    ex.printStackTrace();
}
```

Call Stored Procedure using JDBC (without OUT parameters)

- Stored Procedure - Change price of given book id.

```
DELIMITER //
```

```
CREATE PROCEDURE sp_updateprice(IN p_id INT, IN p_price DOUBLE)
BEGIN
    UPDATE books SET price=p_price WHERE id=p_id;
END;
//
```

```
DELIMITER ;
```

```
CALL sp_updateprice(22, 543.21);
```

- JDBC use CallableStatement interface to invoke the stored procedures.
- CallableStatement interface is extended from PreparedStatement interface.
- Steps to call Stored procedure are same as PreparedStatement.
 - Create connection.
 - Create CallableStatement using con.prepareCall("CALL ...").
 - Set IN parameters using stmt.setXYZ(...);
 - Execute the procedure using stmt.executeQuery() or stmt.executeUpdate().
 - Close statement & connection.
- To invoke stored procedure, in general stmt.execute() is called. This method returns true, if it is returning ResultSet (i.e. multi-row result). Otherwise it returns false, if it is returning update/affected rows count.

```
boolean isResultSet = stmt.execute();
if(isResultSet) {
    ResultSet rs = stmt.getResultSet();
    // process the ResultSet
}
else {
    int count = stmt.getUpdateCount();
    // process the count
}
```

Call Stored Procedure using JDBC (with OUT parameters)

- Stored Procedure - Get title and price of given book id.

```
DELIMITER //
```

```
CREATE PROCEDURE sp_gettitleprice(IN p_id INT, OUT p_name CHAR(40), OUT  
p_price DOUBLE)  
BEGIN  
    SELECT name INTO p_name FROM books WHERE id=p_id;  
    SELECT price INTO p_price FROM books WHERE id=p_id;  
END;  
//
```

```
DELIMITER ;
```

```
CALL sp_gettitleprice(22, @p_name, @p_price);
```

```
SELECT @p_name, @p_price;
```

- Steps to call Stored procedure with out params.
 - Create connection.
 - Create CallableStatement using con.prepareCall("CALL ...").
 - Set IN parameters using stmt.setXYZ(...) and register out parameters using stmt.registerOutParam(...).
 - Execute the procedure using stmt.execute().
 - Get values of out params using stmt.getXYZ(paramNumber).
 - Close statement & connection.